

FutureProof EPM Income from Property Equity

— Technical Architecture

Version 4.5 | Commercial in Confidence

1. EPM Overview

FutureProof transforms home equity into monthly income. We partner with homeowners, borrowers, and capital providers to build a simple system: invest equity, distribute returns, manage risk.

The Four Players

Role	What They Do	Their Stake
Customer	Partners their equity for monthly income	Receives monthly distributions
Investment Partner	Manages the invested capital	Generates ~10% p.a. returns over 30 years
Lender	Originates loans, manages customer relationships	Earns margin (0.75%–2%) on distributions
Wholesale Funder	Supplies capital to lenders	Earns wholesale margin (0.2%–3.6%)

How Monthly Income Works

- Investment Partner's fund earns returns.
- System calculates monthly payout (based on equity %, fund performance, holiday status).
- Customer gets their share.
- Lender deducts margin. Wholesale Funder takes their cut. FutureProof takes 0.25%.

Simple. Repeatable. Auditable.

2. Customer Journey

PHASE 1: Origination (Weeks 1–4)

Customer applies → Deal Agent verifies identity (KYC/AML), values property, assigns risk tier → e-signs documents → Lender & funder approve → Capital transferred → Account live.

PHASE 2: Ongoing (Months 1–30+)

Every month:

- Distribution Agent calculates fund returns.
- Customer gets paid on fixed date.
- Investment Agent watches fund performance. If it drops 10%, interest holiday flag triggers automatically.
- Monthly statement shows distribution, margins, fund health, holiday status.
- Customer can request early exit anytime.

PHASE 3: Exit (Early or Maturity)

Customer requests exit → Deal Agent calculates final amount (fund value minus fees) → Lender approves → Distribution Agent processes final payment → Loan closed → All records archived (7 years).

3. Stakeholder Views & Fraud Monitoring

What each player sees + what we watch for:

Stakeholder	Dashboard Shows	Red Flags
Customer	Monthly distributions, cumulative income, fund chart, holiday status	Multiple withdrawals same day, logins from new locations, rapid device switches
Lender	Distribution schedule, margin received, portfolio health, customer status	Holiday requests that look systematic, unusual pattern shifts, distributions delayed
Investment Partner	Fund performance, ETF data, rebalancing schedule, cost basis	Performance that doesn't match ETFs, holdings mismatches, rebalancing delays
Wholesale Funder	Portfolio yield, default rates, margin collections, lender tier performance	Yield compression, risk concentration, lender financial strain

4. System Architecture

Core Platform

Rails 8.1 + Hotwire + Stimulus

Single monolithic Rails app. Live dashboards update via Hotwire (Turbo). Lightweight JavaScript via Stimulus. No WebSocket complexity.

PostgreSQL

Source of truth: customers, transactions, audit trails. JSON columns for flexible fund metadata. Materialized views for performance aggregation. Database triggers enforce compliance rules automatically. Partitioned by customer ID for scale.

Kafka Event Queue

Agents publish events. Others subscribe. Deal created → Distribution Agent and Compliance Agent react. Distribution executed → Lender gets paid. Holiday triggered → Distribution Agent adjusts calculations. Exactly-once processing with dead-letter queues for failures.

Redis Cache

Sessions, rate limiting, staged calculations, fraud detection (Bloom filters), frequently accessed data.

S3 Storage

Documents, statements, audit logs. Encrypted at rest. Lifecycle: archive after 90 days, delete after 7 years (compliance).

Core Services

Distribution Engine (Solid Queue job)

Runs 1st–5th of month via Rails native background job scheduler. Pulls fund returns, calculates per-customer distributions, stages in Redis, executes ACH/bank transfers via Stripe, publishes events. Retry logic with exponential backoff.

Compliance Engine

Real-time validation against 20+ regulatory rules. Writes immutable audit trail. Database triggers for Change Data Capture.

Fraud Detection

ML pipeline scores every transaction:

- Known patterns (Bloom filter)?
- Behavioral anomalies (isolation forest)?
- Rule violations (impossible geography, velocity)?

Interest Holiday Logic

Monthly job monitors fund performance vs. entry/exit thresholds. Triggers automatically at 90% drop. Releases automatically at 145.8% recovery.

Notification Service

Email (SendGrid), SMS (Twilio), in-app. Stores pending notifications, retries with backoff, confirms delivery.

Document Engine

Server-rendered templates (ERB) + Prawn gem. Generates loan docs, agreements, statements, disclosures. All digitally signed, archived to S3 with immutable version IDs.

5. Four-Agent Architecture

Deal Agent

- Processes applications
- Runs KYC/AML
- Generates documents
- Triggered by: New application

Distribution Agent

- Calculates monthly returns
- Deducts margins
- Executes payments
- Generates statements
- Triggered by: Monthly cutoff date

Investment Agent

- Monitors fund performance
- Calculates rolling metrics
- Decides holiday eligibility
- Triggers/releases holidays
- Triggered by: Monthly performance updates

Compliance Agent

- Real-time fraud detection
- Regulatory rule enforcement
- Generates audit reports
- Flags suspicious activity
- Triggered by: Transaction submission

Agent Communication

All agents publish to Kafka. Others subscribe. Decoupled. Asynchronous. Scalable.

Example flow:

1. Deal Agent publishes `deal.created` .

2. Distribution Agent and Compliance Agent subscribe, wake up.
 3. Distribution Agent publishes `distribution.monthly` .
 4. Lender receives payment.
 5. Investment Agent publishes `holiday.triggered` .
 6. Distribution Agent adjusts calculations.
 7. Compliance Agent publishes `fraud.detected` .
 8. System escalates to lender for review.
-

6. Agent Technology: Claude & Kimi

Why Agents?

Human agents would slow everything down. AI agents process in seconds. Four autonomous agents handle all workflows 24/7.

How We Do It: Claude & Kimi

Claude handles Deal Agent workflows (KYC/AML logic, document generation decisions, eligibility assessment).

Kimi handles Distribution Agent workflows (complex calculation logic, margin deduction rules, payment reconciliation).

Both are integrated via OpenAI-compatible API endpoints that sit inside our private network. **Critical: No customer data ever leaves the system.** All processing happens on-premise. Customer names, account balances, fund performance data, transaction histories—all stay inside our infrastructure. Claude and Kimi receive anonymized, hashed transaction patterns and rule-based queries. They respond with recommendations that Distribution Agent executes, but they never touch raw customer data.

Implementation Details

Rails service classes (e.g., `DealAgent::OnboardingService`). Each agent runs as Solid Queue background job (4 workers per pod). Horizontally scalable.

Event Subscription (Kafka)

Each agent subscribes to relevant Kafka topics using ruby-kafka gem. Consumer groups handle partitioning automatically. Dead-letter queues capture failures.

Idempotency & Retry

Agents implement idempotent handlers. Database unique constraints + transactional locks prevent double-processing. Failed tasks auto-retry (1s → 2s → 4s → 8s → 16s), then escalate.

Distributed Tracing

Every event gets a trace ID. As it flows through agents, each writes logs tagged with that ID. Search CloudWatch by trace ID, see the entire customer journey end-to-end.

State Management

- Deal workflow: `draft → submitted → kyc_pending → approved → funded → active`
- Holiday logic: `normal → holiday_pending → holiday_active → recovery_in_progress → normal`

Call Center (Twilio Flex)

When manual escalation needed, ops team gets a ticket in Twilio Flex. Agent clicks customer, Flex dials via Twilio, logs call recording. If customer calls in, IVR routes to available agent. Agent sees customer profile, recent interactions, account balance.

Sync vs. Async

Async by default (publish event, job queued). Sync only for:

- Stripe payment (5-sec timeout)
- IDology KYC (10-sec timeout)
- Property valuation (15-sec timeout)

All wrapped in circuit breaker pattern.

7. Interest Holiday Mechanism

Why holidays exist: Market downturns shouldn't force customers to pay lenders. We pause interest accrual automatically.

How it works:

Threshold	What Happens
90% drop	Holiday automatically triggers. Customer still gets monthly distributions. Lender gets alert. Statements show holiday status.
145.8% recovery	Holiday automatically lifts. Everything returns to normal.

Example timeline:

- Months 1–10:** Normal operations. Fund does well. Customer gets paid.
- Month 11:** Market drop. Fund hits 88% (below 90% threshold). Holiday flag activates.
- Months 12–18:** Holiday active. Customer still paid. Statements note holiday.
- Month 19:** Fund recovers to 145.8%. Holiday automatically released.

8. Compliance & Fraud Detection

What we check. How we catch bad actors.

Compliance Rules

Rule	What We Verify	What Happens If It Fails
KYC/AML	Customer identity confirmed before onboarding	Application rejected
Responsible Lending	Fund amount appropriate for risk tier	Margin increased; tier adjusted
Distribution Accuracy	Monthly calcs match fund returns	Distribution held pending audit
Fund Performance	Daily returns reconcile with actual	Discrepancy flagged
Holiday Logic	Thresholds applied correctly	Manually reviewed
Data Privacy	PII access logged; encrypted	Unauthorized access alert

Real-Time Fraud Monitoring

Account anomalies:

- Logins from unusual locations
- Rapid device changes
- After-hours patterns

Transaction anomalies:

- Multiple withdrawals same day
- Requests after distribution
- Systematic holiday manipulation

Data inconsistencies:

- Reported vs. actual fund value mismatch
- Distribution calculation errors

Regulatory violations:

- KYC/AML failures
- Undisclosed conflicts
- Unauthorized transfers

Audit & Reporting

Audit Trail: Every transaction, margin payment, holiday trigger recorded with timestamp, user ID, changes. Retained 7 years minimum.

Monthly Reporting: Lenders and wholesale funders get automated reports on portfolio health, margin collections, default scenarios.

Annual Certification: External auditor reviews all regulatory requirements (Australia, UK, US).

9. Technology Stack

Backend

- Rails 8.1 (Ruby 3.4.8)
- Solid Queue (native Rails background jobs, 4 workers)
- ruby-kafka (event streaming)
- Puma (web server, 3 threads, auto-scaling)

Data

- PostgreSQL 16 (primary + read replica)
- Redis 7 (cache + session store)
- Kafka 3.6 (event log, 3 replicas)
- PgBouncer (connection pooling)

Agents & Workflow

- Solid Queue (async execution)
- Kafka (pub/sub, 4 topic streams)
- OpenTelemetry (distributed tracing)
- Dead-letter queues (failure handling)
- State machines (holiday logic, deal workflow)

Communication

- SendGrid (email)
- Twilio (SMS + voice)
- Twilio Flex (call center)
- WebRTC (video calls)
- DocuSign (e-signature)

Frontend

- Hotwire (Turbo Streams + Drive)
- Stimulus JS (lightweight controllers)
- Tailwind CSS (styling)
- ERB templates (server-rendered)
- Chart.js (performance charts)

Infrastructure

- AWS S3 (documents, versioned)
- AWS CloudFront (CDN)
- S3 Glacier (archive)

- Fly.io (deployment)
- GitHub Actions (CI/CD)
- Docker (containers)
- Terraform (IaC)

Monitoring

- Sentry (error tracking)
- CloudWatch (logs)
- Grafana (dashboards)
- PagerDuty (escalation)
- HTTPS/TLS, WAF, CSP headers

10. Database Schema

Core tables:

Table	Purpose	Key Fields
customers	Homeowners	id, email, phone, kyc_status, aml_status, risk_tier
properties	Residential with equity	id, customer_id, address, property_value, equity_percentage, loan_amount
accounts	Customer investment accounts	id, customer_id, property_id, starting_balance, current_balance, holiday_flagged
fund_performance	Monthly metrics (materialized view)	account_id, month, starting_value, ending_value, return_pct, rolling_12m_return
distributions	Monthly payouts (immutable)	id, account_id, fund_return_amount, lender_margin, customer_payout, executed_at
payments	ACH/bank transfers (immutable)	id, distribution_id, amount, bank_account_id, status, stripe_charge_id
audit_log	Append-only event log	id, entity_type, action, changes_json, user_id, timestamp, trace_id
fraud_alerts	Detected patterns	id, customer_id, alert_type, fraud_score, status, reviewed_by
lenders	Loan originators	id, name, tier, max_funding_line, current_balance, margin_rate
wholesale_funders	Capital providers	id, name, funding_capacity, current_deployed, margin_rate

Design patterns:

- **Immutable logs:** `distributions` , `payments` , `audit_log` have no UPDATE/DELETE. All changes are INSERT-only with timestamps.
- **Soft deletes:** `customers` , `accounts` include `deleted_at` . Preserves history; queries filter `WHERE deleted_at IS NULL` .
- **JSON columns:** `audit_log.changes_json` stores before/after. `fraud_alerts.details_json` stores model output. `accounts.metadata_json` stores region-specific terms.
- **Materialized views:** `fund_performance` refreshed nightly. Improves dashboard load times.
- **Partitioning:** `distributions` , `payments` partitioned by `created_at` (monthly partitions). Old partitions move to cold storage.

11. Deployment & Quality Assurance

Testing

- **Unit tests:** Core logic (distributions, holiday logic, fraud rules).
- **Integration tests:** Agent workflows, end-to-end customer journeys, payments.
- **Performance tests:** Batch processing at scale (1000s of customers), query optimization.
- **Security tests:** Auth, authorization, data access, PII encryption.
- **Manual tests:** Customer portal UX, lender reporting, admin workflows.

Continuous Deployment

GitHub Actions runs tests on every PR. RSpec (200+ specs), Rubocop (linting), Brakeman (security), database migrations validated. Tests + code review approval required before merge.

Staging environment deployed automatically from main branch using Fly.io (same infra as production). Runs against production-replica DB (encrypted, daily sanitized PII). Manual smoke tests.

Production: Blue-green deployment on Fly.io. Traffic shifted 10% → 50% → 100% over 10 minutes (canary rollout). Health checks verify Rails boot, database, Kafka, latency. Automatic rollback if health checks fail within 5 minutes.

Monitoring & Alerting

- **Sentry:** Real-time errors. Critical errors trigger PagerDuty within 2 minutes.
- **CloudWatch:** Structured JSON logging (trace_id, user_id, duration_ms, queries, API calls). 90 days hot, then S3 archive.
- **Grafana:** System health (CPU, memory, disk), app metrics (requests/sec, error rate, p95 latency), business metrics (distributions, revenue), agent health (lag, dead-letter size), database health (pool, replication lag).
- **Alerts:** error rate >1%, distribution latency >60s, Kafka lag >1000 messages.

Commercial in Confidence

© 2026 FutureProof Financial Group Limited. All rights reserved.